

Advanced Python Programming

Unit 2 & 4 — Django, GUI & MySQL | Assignment No. 3

Section A: Django Web Projects (Q1–Q17)

Q1. Create simple web project in Django to display Hello World message.

File: myproject/urls.py

```
from django.contrib import admin
from django.urls import path
from . import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.hello_world, name='hello'),
]
```

File: myproject/views.py

```
from django.http import HttpResponse

def hello_world(request):
    return HttpResponse("<h1>Hello, World!</h1>")
```

File: Setup Commands

```
django-admin startproject myproject
cd myproject
python manage.py runserver
# Visit: http://127.0.0.1:8000/
```

Output: Browser displays: Hello, World! (as heading)

Note: Create views.py inside the project folder. Add it to urls.py.

Q2. Create simple web project in Django to demonstrate the use of HttpResponse.

File: views.py

```
from django.http import HttpResponse
import datetime

def home(request):
    return HttpResponse("<h2>Welcome to Django!</h2>")

def date_view(request):
    now = datetime.datetime.now()
    html = f"<h3>Current Date & Time: {now.strftime('%d-%m-%Y %H:%M:%S')}</h3>"
    return HttpResponse(html)

def custom_status(request):
    # HttpResponse with custom status code
    return HttpResponse("<h3>Resource Created!</h3>", status=201)

def styled_response(request):
    html = (
        "<html><body style='background:#e3f2fd;font-family:Arial'>"
        "<h1 style='color:#1a237e'>HttpResponse Demo</h1>"
        "<p>This response was built using HttpResponse object.</p>"
        "</body></html>"
    )
    return HttpResponse(html, content_type='text/html')
```

File: urls.py

```
from django.urls import path
from . import views

urlpatterns = [
```

```

path('', views.home, name='home'),
path('date/', views.date_view, name='date'),
path('create/', views.custom_status, name='create'),
path('styled/', views.styled_response, name='styled'),
]

```

Output: / -> Welcome to Django! /date/ -> Current Date & Time /styled/ -> Styled page

Q3. Create simple web project in Django to demonstrate the render method.

File: views.py

```

from django.shortcuts import render

def home(request):
    context = {
        'title' : 'Render Method Demo',
        'message': 'Django render() passes data to templates!',
        'items' : ['Python', 'Django', 'HTML', 'CSS'],
    }
    return render(request, 'home.html', context)

```

File: templates/home.html

```

<!DOCTYPE html>
<html>
<head><title>{{ title }}</title></head>
<body style="font-family:Arial; padding:30px; background:#e8f5e9">
  <h1 style="color:#1b5e20">{{ title }}</h1>
  <p>{{ message }}</p>
  <h3>Items List:</h3>
  <ul>
    {% for item in items %}
      <li>{{ item }}</li>
    {% endfor %}
  </ul>
</body>
</html>

```

File: settings.py — TEMPLATES setting

```

TEMPLATES = [{
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
    'DIRS': [BASE_DIR / 'templates'],  # <-- add this
    ...
}]

```

Output: Browser shows: Title, Message, and a bullet list: Python, Django, HTML, CSS

Q4. Create web project "COLLEGE" with app "DEPT". Design 3 URL patterns to show department info.

File: Setup

```

django-admin startproject COLLEGE
cd COLLEGE
python manage.py startapp DEPT

```

File: DEPT/views.py

```

from django.http import HttpResponse
from django.shortcuts import render

DEPARTMENTS = {
    'cs' : {'name':'Computer Science', 'hod':'Dr. Mehta', 'students':240},
    'mech' : {'name':'Mechanical', 'hod':'Dr. Patil', 'students':180},
    'civil': {'name':'Civil Engineering', 'hod':'Dr. Sharma', 'students':160},
}

def dept_list(request):
    return render(request, 'dept_list.html', {'departments': DEPARTMENTS})

def dept_detail(request, dept_code):

```

```

dept = DEPARTMENTS.get(dept_code)
if dept:
    return render(request, 'dept_detail.html', {'dept': dept, 'code': dept_code})
return HttpResponse("<h3>Department not found!</h3>", status=404)

```

```

def dept_students(request, dept_code):
    dept = DEPARTMENTS.get(dept_code)
    count = dept['students'] if dept else 0
    return HttpResponse(f"<h2>{dept_code.upper()} has {count} students.</h2>")

```

File: COLLEGE/urls.py

```

from django.urls import path, include
urlpatterns = [
    path('dept/', include('DEPT.urls')),
]

```

File: DEPT/urls.py

```

from django.urls import path
from . import views

urlpatterns = [
    path('', views.dept_list, name='dept_list'),
    path('<str:dept_code>/', views.dept_detail, name='dept_detail'),
    path('<str:dept_code>/stu/', views.dept_students, name='dept_students'),
]

```

File: templates/dept_list.html

```

<!DOCTYPE html><html><body style="font-family:Arial;padding:20px">
<h2>College Departments</h2>
<ul>
    {% for code, info in departments.items %}
        <li><a href="/dept/{{ code }}">{{ info.name }}</a> – HOD: {{ info.hod }}</li>
    {% endfor %}
</ul>
</body></html>

```

Output: /dept/ -> list of 3 departments /dept/cs/ -> CS details /dept/cs/stu/ -> student count

Q5. Create web project with multiple apps which contain Index, Home, Data pages.

File: Setup

```

django-admin startproject multiapp
cd multiapp
python manage.py startapp pages
python manage.py startapp data

```

File: pages/views.py

```

from django.shortcuts import render

def index(request):
    return render(request, 'pages/index.html',
        {'title': 'Index Page', 'msg': 'Welcome to Our Site!'})

def home(request):
    features = ['Fast Performance', 'Secure', 'Scalable', 'Easy to Use']
    return render(request, 'pages/home.html',
        {'title': 'Home Page', 'features': features})

```

File: data/views.py

```

from django.shortcuts import render

def data_page(request):
    students = [
        {'name': 'Rahul', 'branch': 'CS', 'marks': 87},
        {'name': 'Priya', 'branch': 'ECE', 'marks': 92},
        {'name': 'Amit', 'branch': 'Mech', 'marks': 78},
    ]

```

```
return render(request, 'data/data.html', {'students': students})
```

File: multiapp/urls.py

```
from django.urls import path
from pages import views as pv
from data import views as dv

urlpatterns = [
    path('', pv.index, name='index'),
    path('home/', pv.home, name='home'),
    path('data/', dv.data_page, name='data'),
]
```

File: templates/data/data.html (key part)

```
<table border="1" style="border-collapse: collapse; width: 100%">
  <tr><th>Name</th><th>Branch</th><th>Marks</th></tr>
  {% for s in students %}
  <tr>
    <td>{{ s.name }}</td><td>{{ s.branch }}</td><td>{{ s.marks }}</td>
  </tr>
  {% endfor %}
</table>
```

Output: / -> Index Page /home/ -> Home with features list /data/ -> Student data table

Q6. Create web project with static files (CSS, JS, Image).

File: Project structure

```
staticproject/
  static/
    css/    style.css
    js/     script.js
    img/    logo.png
  templates/ index.html
  settings.py
```

File: settings.py additions

```
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
```

File: templates/index.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <title>Static Files Demo</title>
  <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
  <div class="container">
    
    <h1 id="heading">Static Files in Django</h1>
    <p class="subtitle">CSS, JS and Images loaded via {% verbatim %}{% static %}{% endverbatim %} tag</p>
    <button onclick="greet()">Click Me</button>
  </div>
  <script src="{% static 'js/script.js' %}"></script>
</body>
</html>
```

File: static/css/style.css

```
body { font-family: Arial; background: #e8eaf6; margin: 0; }
.container { max-width: 700px; margin: 60px auto; text-align: center;
  background: white; padding: 40px; border-radius: 12px;
  box-shadow: 0 4px 20px rgba(0,0,0,0.1); }
h1 { color: #1a237e; }
.subtitle { color: #555; }
button { background: #1a237e; color: white; padding: 10px 24px; }
```

```
border:none; border-radius:6px; cursor:pointer; font-size:15px; }
```

File: static/js/script.js

```
function greet() {  
    alert('Hello from Django Static JS!');  
    document.getElementById('heading').style.color = '#b71c1c';  
}
```

Output: Browser shows styled page with logo image, heading in blue, and a clickable JS button.

Q7. Create web project to demonstrate simple file upload.

File: views.py

```
from django.shortcuts import render  
from django.http import HttpResponse  
import os  
  
def upload_file(request):  
    if request.method == 'POST' and request.FILES.get('myfile'):  
        f = request.FILES['myfile']  
        upload_dir = 'uploads/'  
        os.makedirs(upload_dir, exist_ok=True)  
        filepath = os.path.join(upload_dir, f.name)  
        with open(filepath, 'wb+') as dest:  
            for chunk in f.chunks():  
                dest.write(chunk)  
        info = {  
            'name': f.name, 'size': f.size,  
            'type': f.content_type, 'path': filepath,  
        }  
        return render(request, 'upload.html', {'success': True, 'info': info})  
    return render(request, 'upload.html')
```

File: templates/upload.html

```
<!DOCTYPE html><html><body style="font-family:Arial;padding:30px;background:#f5f5f5">  
<h2>File Upload Demo</h2>  
{% if success %}  
    <div style="background:#e8f5e9;padding:15px;border-radius:8px">  
        <h3>Upload Successful!</h3>  
        <p>Name: {{ info.name }}</p>  
        <p>Size: {{ info.size }} bytes</p>  
        <p>Type: {{ info.type }}</p>  
    </div>  
{% endif %}  
<form method="post" enctype="multipart/form-data">  
    {% csrf_token %}  
    <input type="file" name="myfile" required><br><br>  
    <button type="submit" style="background:#1a237e;color:white;padding:8px 20px">  
        Upload File  
    </button>  
</form>  
</body></html>
```

Note: enctype='multipart/form-data' is required for file uploads. request.FILES handles uploaded files.

Output: /upload/ -> Form to select file. After upload: shows filename, size, type.

Q8. Create web project to demonstrate admin panel.

File: models.py

```
from django.db import models  
  
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    branch = models.CharField(max_length=50)  
    semester = models.IntegerField()  
    percentage = models.FloatField()
```

```

email      = models.EmailField(unique=True)
created_at = models.DateTimeField(auto_now_add=True)

def __str__(self):
    return f"{self.name} ({self.branch})"

class Meta:
    ordering = ['-percentage']

```

File: admin.py

```

from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ('name', 'branch', 'semester', 'percentage', 'email')
    list_filter = ('branch', 'semester')
    search_fields = ('name', 'email')
    ordering = ('-percentage',)

```

File: Setup Commands

```

python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser # set username/password
python manage.py runserver
# Visit: http://127.0.0.1:8000/admin/

```

Output: /admin/ -> Login with superuser credentials -> Manage Students with search, filter, sort

Q9. Create web project by using Django template TAGS.

File: views.py

```

from django.shortcuts import render
from datetime import date

def template_tags_demo(request):
    context = {
        'name'      : 'Rahul',
        'score'     : 82,
        'is_admin'  : False,
        'fruits'    : ['Apple', 'Mango', 'Banana', 'Orange'],
        'today'     : date.today(),
        'bio'       : 'django web framework python',
        'price'     : 1234.567,
    }
    return render(request, 'tags.html', context)

```

File: templates/tags.html

```

{% load static %}
<!DOCTYPE html>
<html><body style="font-family:Arial; padding:20px">
<h2>Django Template Tags Demo</h2>

<!-- Variable tag -->
<p>Hello, {{ name }}!</p>

<!-- if / elif / else tag -->
{% if score >= 90 %}
    <p>Grade: A (Distinction)</p>
{% elif score >= 75 %}
    <p>Grade: B (First Class)</p>
{% else %}
    <p>Grade: C (Pass)</p>
{% endif %}

<!-- for loop tag with forloop counter -->

```

```

<ul>
{% for fruit in fruits %}
    <li>{{ forloop.counter }}. {{ fruit }}</li>
{% empty %}
    <li>No fruits found.</li>
{% endfor %}
</ul>

<!-- Filters -->
<p>Upper: {{ name|upper }}</p>
<p>Length of fruits: {{ fruits|length }}</p>
<p>Price formatted: {{ price|floatformat:2 }}</p>
<p>Today: {{ today|date:"d M Y" }}</p>
<p>Truncated bio: {{ bio|truncatewords:2 }}</p>
<p>Title case: {{ bio|title }}</p>

<!-- with tag -->
{% with total=fruits|length %}
    <p>Total fruits: {{ total }}</p>
{% endwith %}

<!-- comment tag -->
{% comment %}This is a Django comment - not rendered{% endcomment %}

<!-- url tag -->
<a href="{% url 'tags_demo' %}">Reload Page</a>
</body></html>

```

Output: Hello, Rahul! Grade: B (First Class) 1.Apple 2.Mango 3.Banana 4.Orange Upper: RAHUL Length: 4 Price: 1234.57 Today: 08 Mar 2026 Truncated: django web...

Q10. Create web project to register model in admin panel.

File: models.py

```

from django.db import models

class Department(models.Model):
    name = models.CharField(max_length=100)
    code = models.CharField(max_length=10, unique=True)
    hod = models.CharField(max_length=100)
    capacity = models.IntegerField(default=60)

    def __str__(self):
        return f"{self.code} - {self.name}"

class Book(models.Model):
    title = models.CharField(max_length=200)
    author = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=8, decimal_places=2)
    available = models.BooleanField(default=True)

    def __str__(self):
        return self.title

```

File: admin.py

```

from django.contrib import admin
from .models import Department, Book

# Method 1: Simple registration
admin.site.register(Book)

# Method 2: Using ModelAdmin class
class DepartmentAdmin(admin.ModelAdmin):
    list_display = ('code', 'name', 'hod', 'capacity')

```

```
list_filter    = ('hod',)
search_fields  = ('name', 'code')
list_editable  = ('capacity',)
```

```
admin.site.register(Department, DepartmentAdmin)
```

```
# Customize admin site branding
admin.site.site_header = "College Admin Portal"
admin.site.site_title   = "College Admin"
admin.site.index_title  = "Welcome to College Management"
```

Output: /admin/ shows 'College Admin Portal' with Department and Book models. Both fully manageable.

Q11. Create web project with multiple templates (insert data, display list, display table).

File: models.py

```
from django.db import models
class Student(models.Model):
    name      = models.CharField(max_length=100)
    branch    = models.CharField(max_length=50)
    marks     = models.FloatField()
    def __str__(self): return self.name
```

File: views.py

```
from django.shortcuts import render, redirect
from .models import Student
from django.contrib import messages

def insert_student(request):
    if request.method == 'POST':
        Student.objects.create(
            name = request.POST['name'],
            branch= request.POST['branch'],
            marks = request.POST['marks'],
        )
        messages.success(request, 'Student added successfully!')
        return redirect('student_list')
    return render(request, 'insert.html')

def student_list(request):
    students = Student.objects.all()
    return render(request, 'list.html', {'students': students})

def student_table(request):
    students = Student.objects.all().order_by('-marks')
    avg = sum(s.marks for s in students) / len(students) if students else 0
    return render(request, 'table.html', {'students': students, 'avg': avg})
```

File: templates/insert.html

```
<form method="post" style="padding:20px;font-family:Arial">
    {% csrf_token %}
    <h2>Add Student</h2>
    Name:   <input name="name"   type="text"   required><br><br>
    Branch: <input name="branch" type="text"   required><br><br>
    Marks:  <input name="marks"  type="number" step="0.1" required><br><br>
    <button type="submit">Save Student</button>
</form>
```

File: templates/table.html (key part)

```
<table border="1" style="border-collapse:collapse;width:100%;font-family:Arial">
    <tr style="background:#1a237e;color:white">
        <th>#</th><th>Name</th><th>Branch</th><th>Marks</th><th>Grade</th>
    </tr>
    {% for s in students %}
    <tr>
```



```

<td>{{ forloop.counter }}</td>
<td>{{ s.name }}</td>
<td>{{ s.branch }}</td>
<td>{{ s.marks }}</td>
<td>{% if s.marks >= 90 %}A{% elif s.marks >= 75 %}B{% else %}C{% endif %}</td>
</tr>
{% endfor %}
<tr><td colspan="3"><b>Average</b></td><td>{{ avg|floatformat:2 }}</td></tr>
</table>

```

Output: /add/ -> Insert form /list/ -> Card list view /table/ -> Sorted table with grades & average

Q12. Create CRUD web project for student registration.

File: models.py

```

from django.db import models

class Student(models.Model):
    name          = models.CharField(max_length=100)
    roll_no       = models.CharField(max_length=20, unique=True)
    branch        = models.CharField(max_length=50)
    semester      = models.IntegerField()
    percentage     = models.FloatField()
    email         = models.EmailField(unique=True)
    def __str__(self): return f"{self.roll_no} - {self.name}"

```

File: views.py

```

from django.shortcuts import render, get_object_or_404, redirect
from .models import Student

def student_list(request):
    students = Student.objects.all()
    return render(request, 'crud/list.html', {'students': students})

def student_create(request):
    if request.method == 'POST':
        Student.objects.create(**{k: request.POST[k]
                                   for k in ['name', 'roll_no', 'branch', 'semester', 'percentage', 'email']})
        return redirect('student_list')
    return render(request, 'crud/form.html', {'action': 'Add'})

def student_update(request, pk):
    stu = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        for field in ['name', 'branch', 'semester', 'percentage', 'email']:
            setattr(stu, field, request.POST[field])
        stu.save()
        return redirect('student_list')
    return render(request, 'crud/form.html', {'student': stu, 'action': 'Update'})

def student_delete(request, pk):
    stu = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        stu.delete()
        return redirect('student_list')
    return render(request, 'crud/confirm_delete.html', {'student': stu})

```

File: urls.py

```

from django.urls import path
from . import views

urlpatterns = [
    path('', views.student_list, name='student_list'),
    path('add/', views.student_create, name='student_create'),
    path('edit/<int:pk>', views.student_update, name='student_update'),
    path('del/<int:pk>', views.student_delete, name='student_delete'),

```

```
]
```

File: templates/crud/list.html (key part)

```
{% for s in students %}
<tr>
  <td>{{ s.roll_no }}</td><td>{{ s.name }}</td>
  <td>{{ s.branch }}</td><td>{{ s.percentage }}</td>
  <td>
    <a href="{% url 'student_update' s.pk %}">Edit</a> |
    <a href="{% url 'student_delete' s.pk %}">Delete</a>
  </td>
</tr>
{% endfor %}
```

Output: Full CRUD: / -> list with Edit/Delete links /add/ -> form /edit/1/ -> update /del/1/ -> confirm delete

Q13. Create web project to demonstrate authentication and authorization.

File: views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required, permission_required
from django.contrib import messages

def login_view(request):
    if request.method == 'POST':
        user = authenticate(request,
                             username=request.POST['username'],
                             password=request.POST['password'])
        if user:
            login(request, user)
            return redirect('dashboard')
        messages.error(request, 'Invalid credentials!')
    return render(request, 'auth/login.html')

def logout_view(request):
    logout(request)
    return redirect('login')

@login_required(login_url='/login/')
def dashboard(request):
    return render(request, 'auth/dashboard.html', {'user': request.user})

@login_required(login_url='/login/')
def profile(request):
    return render(request, 'auth/profile.html', {'user': request.user})

# Admin-only view (authorization)
@login_required(login_url='/login/')
def admin_panel(request):
    if not request.user.is_staff:
        return render(request, 'auth/forbidden.html', status=403)
    return render(request, 'auth/admin_panel.html')
```

File: templates/auth/login.html

```
<form method="post" style="max-width:350px;margin:80px auto;font-family:Arial">
  {% csrf_token %}
  <h2>Login</h2>
  {% if messages %}
    {% for msg in messages %}
      <p style="color:red">{{ msg }}</p>
    {% endfor %}
  {% endif %}
  <input name="username" placeholder="Username" style="width:100%;padding:8px;margin:6px 0"><br>
```

```



```

File: urls.py

```

urlpatterns = [
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
    path('dashboard/', views.dashboard, name='dashboard'),
    path('admin-panel/', views.admin_panel, name='admin_panel'),
]

```

Output: /login/ -> form /dashboard/ -> redirects to login if not authenticated /admin-panel/ -> 403 for non-staff

Q14. Create web project to demonstrate Django forms.

File: forms.py

```

from django import forms
from django.core.validators import MinValueValidator, MaxValueValidator

class StudentForm(forms.Form):
    name = forms.CharField(max_length=100,
                           widget=forms.TextInput(attrs={'placeholder':'Full Name'}))
    email = forms.EmailField()
    branch = forms.ChoiceField(choices=[
        ('CS', 'Computer Science'), ('ECE', 'Electronics'),
        ('ME', 'Mechanical'), ('CE', 'Civil')])
    semester = forms.IntegerField(validators=[MinValueValidator(1),
                                              MaxValueValidator(8)])
    marks = forms.FloatField(validators=[MinValueValidator(0),
                                         MaxValueValidator(100)])
    agree = forms.BooleanField(label='I agree to terms',
                              required=True)

    def clean_name(self):
        name = self.cleaned_data['name']
        if len(name) < 3:
            raise forms.ValidationError("Name must be at least 3 characters.")
        return name.title()

```

File: views.py

```

from django.shortcuts import render
from .forms import StudentForm

def student_form(request):
    if request.method == 'POST':
        form = StudentForm(request.POST)
        if form.is_valid():
            data = form.cleaned_data
            return render(request, 'form_success.html', {'data': data})
    else:
        form = StudentForm()
    return render(request, 'student_form.html', {'form': form})

```

File: templates/student_form.html

```

<form method="post" style="max-width:500px;margin:40px auto;font-family:Arial">
    {% csrf_token %}
    <h2>Student Form (Django Forms Demo)</h2>
    {% for field in form %}
        <div style="margin:10px 0">
            {{ field.label_tag }}
            {{ field }}
            {% if field.errors %}

```

```

        <span style="color:red">{{ field.errors.0 }}</span>
    {% endif %}
</div>
{% endfor %}
<button type="submit" style="background:#1a237e;color:white;padding:8px 20px">Submit</button>
</form>

```

Output: Form renders with validation. Invalid input shows inline errors. Valid submit shows success page with cleaned_data.

Q15. Create web project to demonstrate Django Model forms.

File: models.py

```

from django.db import models

class Employee(models.Model):
    name = models.CharField(max_length=100)
    department = models.CharField(max_length=50)
    designation = models.CharField(max_length=50)
    salary = models.DecimalField(max_digits=10, decimal_places=2)
    email = models.EmailField(unique=True)
    join_date = models.DateField()
    def __str__(self): return self.name

```

File: forms.py

```

from django import forms
from .models import Employee

class EmployeeModelForm(forms.ModelForm):
    class Meta:
        model = Employee
        fields = '__all__' # Include all model fields
        widgets = {
            'join_date': forms.DateInput(attrs={'type': 'date'}),
            'salary': forms.NumberInput(attrs={'step': '0.01'}),
        }
        labels = {
            'join_date': 'Date of Joining',
        }

    def clean_salary(self):
        salary = self.cleaned_data['salary']
        if salary < 10000:
            raise forms.ValidationError("Salary must be at least Rs.10,000")
        return salary

```

File: views.py

```

from django.shortcuts import render, redirect
from .forms import EmployeeModelForm
from .models import Employee

def add_employee(request):
    form = EmployeeModelForm(request.POST or None)
    if form.is_valid():
        form.save() # Directly saves to DB (ModelForm advantage!)
        return redirect('emp_list')
    return render(request, 'emp_form.html', {'form': form})

def emp_list(request):
    employees = Employee.objects.all()
    return render(request, 'emp_list.html', {'employees': employees})

```

Output: ModelForm auto-generates fields from model. form.save() directly inserts into DB without manual extraction.

Note: Key advantage: ModelForm eliminates boilerplate — no need to manually map form fields to model fields.

Q16. Create web project to demonstrate session and cookies.

File: views.py

```
from django.shortcuts import render, redirect
from django.http import HttpResponseRedirect

# ■■ SESSION DEMO ■■
def set_session(request):
    request.session['username'] = 'Rahul'
    request.session['branch'] = 'CS'
    request.session['login_count'] = request.session.get('login_count', 0) + 1
    return redirect('get_session')

def get_session(request):
    username = request.session.get('username', 'Guest')
    branch = request.session.get('branch', 'N/A')
    login_count = request.session.get('login_count', 0)
    return render(request, 'session.html', {
        'username': username, 'branch': branch, 'count': login_count})

def clear_session(request):
    request.session.flush() # Destroy entire session
    return redirect('get_session')

# ■■ COOKIE DEMO ■■
def set_cookie(request):
    response = redirect('get_cookie')
    response.set_cookie('theme', 'dark', max_age=3600)
    response.set_cookie('language', 'en', max_age=3600)
    response.set_cookie('visit_count',
        str(int(request.COOKIES.get('visit_count', 0)) + 1),
        max_age=86400)
    return response

def get_cookie(request):
    theme = request.COOKIES.get('theme', 'light')
    language = request.COOKIES.get('language', 'en')
    visit_count = request.COOKIES.get('visit_count', '1')
    return render(request, 'cookie.html', {
        'theme': theme, 'language': language, 'visits': visit_count})

def delete_cookie(request):
    response = redirect('get_cookie')
    response.delete_cookie('theme')
    response.delete_cookie('language')
    return response
```

Output: /set-session/ -> stores data in server session /get-session/ -> shows: Welcome Rahul (CS) | Login count: 3 /set-cookie/ -> stores theme, language in browser cookie /get-cookie/ -> shows: Theme: dark | Language: en | Visits: 5

Note: Sessions store data server-side (secure). Cookies store data client-side (browser). max_age sets expiry in seconds.

Q17. Create web project to demonstrate authentication and authorization in REST framework.

File: Install

```
pip install djangorestframework djangorestframework-simplejwt
```

File: settings.py

```
INSTALLED_APPS = [..., 'rest_framework', 'rest_framework_simplejwt']

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
```

```

        'rest_framework.permissions.IsAuthenticated',
    ],
}

```

File: serializers.py

```

from rest_framework import serializers
from .models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = '__all__'

```

File: views.py

```

from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated, IsAdminUser, AllowAny
from rest_framework_simplejwt.tokens import RefreshToken
from django.contrib.auth import authenticate
from .models import Student
from .serializers import StudentSerializer

class LoginView(APIView):
    permission_classes = [AllowAny]
    def post(self, request):
        user = authenticate(username=request.data.get('username'),
                             password=request.data.get('password'))
        if user:
            refresh = RefreshToken.for_user(user)
            return Response({
                'refresh': str(refresh),
                'access' : str(refresh.access_token),
                'user'   : user.username,
            })
        return Response({'error': 'Invalid credentials'}, status=401)

class StudentListView(APIView):
    permission_classes = [IsAuthenticated] # Any logged-in user
    def get(self, request):
        students = Student.objects.all()
        return Response(StudentSerializer(students, many=True).data)

class AdminOnlyView(APIView):
    permission_classes = [IsAdminUser] # Only staff/admin
    def delete(self, request, pk):
        Student.objects.filter(pk=pk).delete()
        return Response({'message': f'Student {pk} deleted.'})

```

File: urls.py

```

from django.urls import path
from rest_framework_simplejwt.views import TokenRefreshView
from . import views

urlpatterns = [
    path('api/login/', views.LoginView.as_view()),
    path('api/token/refresh/', TokenRefreshView.as_view()),
    path('api/students/', views.StudentListView.as_view()),
    path('api/students/<int:pk>/delete/', views.AdminOnlyView.as_view()),
]

```

Output: POST /api/login/ -> returns {access, refresh} JWT tokens GET /api/students/ -> returns student list (requires Bearer token in header) DELETE /api/students/1/delete/ -> admin only (403 for regular users)

Section B: Python Basics & GUI (Q18–Q25)

Q18. Write a Python program to create a list and perform append, insert, delete.

```
# Create a list
fruits = ['Apple', 'Mango', 'Banana']
print(f"Original list : {fruits}")

# append() - adds to END
fruits.append('Orange')
print(f"After append() : {fruits}")

# insert(index, item) - inserts at position
fruits.insert(1, 'Grapes')
print(f"After insert(1) : {fruits}")

# insert at beginning
fruits.insert(0, 'Kiwi')
print(f"After insert(0) : {fruits}")

# remove() - removes by VALUE
fruits.remove('Banana')
print(f"After remove() : {fruits}")

# pop() - removes by INDEX (default: last)
popped = fruits.pop()
print(f"After pop()      : {fruits} [popped: {popped}]")

# pop at specific index
popped2 = fruits.pop(1)
print(f"After pop(1)     : {fruits} [popped: {popped2}]")

# del keyword
del fruits[0]
print(f"After del[0]      : {fruits}")

# clear() - removes all
temp = ['a', 'b', 'c']
temp.clear()
print(f"After clear()     : {temp}")

# Useful list methods
nums = [4, 2, 8, 1, 9, 3]
nums.sort()
print(f"After sort()      : {nums}")
nums.reverse()
print(f"After reverse() : {nums}")
print(f"Index of 8         : {nums.index(8)}")
print(f"Count of 4         : {nums.count(4)}")
```

```
Output: Original list : ['Apple', 'Mango', 'Banana'] After append() : ['Apple', 'Mango', 'Banana',
'Orange'] After insert(1): ['Apple', 'Grapes', 'Mango', 'Banana', 'Orange'] After insert(0): ['Kiwi',
'Apple', 'Grapes', 'Mango', 'Banana', 'Orange'] After remove() : ['Kiwi', 'Apple', 'Grapes', 'Mango',
'Orange'] After pop() : ['Kiwi', 'Apple', 'Grapes', 'Mango'] [popped: Orange] After pop(1) : ['Kiwi',
'Grapes', 'Mango'] [popped: Apple] After del[0] : ['Grapes', 'Mango'] After clear() : [] After sort()
: [1, 2, 3, 4, 8, 9] After reverse(): [9, 8, 4, 3, 2, 1] Index of 8 : 1 Count of 4: 1
```

Q19. Create and display a student grade dictionary (rollno, grade).

```
# Create student grade dictionary
student_grades = {
    'R001': {'name': 'Rahul', 'marks': 92, 'grade': 'A'},
    'R002': {'name': 'Priya', 'marks': 85, 'grade': 'B'},
    'R003': {'name': 'Amit', 'marks': 73, 'grade': 'C'},
    'R004': {'name': 'Sneha', 'marks': 95, 'grade': 'A+'},
```

```

    'R005': {'name': 'Rohan', 'marks': 60, 'grade': 'D'},
}

# Display all records
print(f"{'Roll No':<8} {'Name':<12} {'Marks':<8} {'Grade'}")
print("-" * 38)
for roll, info in student_grades.items():
    print(f"{'roll':<8} {'info['name']':<12} {'info['marks']':<8} {'info['grade']}")

# Access specific student
print(f"\nR002's grade: {student_grades['R002']['grade']}")

# Add new student
student_grades['R006'] = {'name': 'Pooja', 'marks': 88, 'grade': 'B+'}
print(f"After adding R006: {len(student_grades)} students")

# Update grade
student_grades['R003']['grade'] = 'B'
print(f"R003 updated grade: {student_grades['R003']['grade']}")

# Delete entry
del student_grades['R005']
print(f"After deleting R005: {len(student_grades)} students")

# Dictionary methods
print(f"All roll numbers: {list(student_grades.keys())}")
toppers = {r: i for r, i in student_grades.items() if i['marks'] >= 85}
print(f"Toppers (>=85 marks):")
for r, i in toppers.items():
    print(f"    {r}: {i['name']} - {i['marks']}")

```

```

Output: Roll No Name Marks Grade ----- R001 Rahul 92 A R002 Priya 85
B R003 Amit 73 C R004 Sneha 95 A+ R005 Rohan 60 D R002's grade: B After adding R006: 6 students R003
updated grade: B After deleting R005: 5 students All roll numbers: ['R001', 'R002', 'R003', 'R004',
'R006'] Toppers: R001:Rahul-92 R002:Priya-85 R004:Sneha-95 R006:Pooja-88

```

Q20. Create a basic login form using Entry and Button.

```

import tkinter as tk
from tkinter import messagebox

VALID_USERS = {'admin': 'admin123', 'student': 'pass456', 'teacher': 'teach789'}

def login():
    uid = user_entry.get().strip()
    pwd = pass_entry.get()
    if not uid or not pwd:
        messagebox.showwarning("Incomplete", "Please enter both fields!")
        return
    if uid in VALID_USERS and VALID_USERS[uid] == pwd:
        messagebox.showinfo("Success", f"Welcome, {uid}!")
        root.destroy()
    else:
        attempts[0] += 1
        messagebox.showerror("Failed", f"Invalid credentials! Attempt {attempts[0]}/3")
        pass_entry.delete(0, tk.END)
        if attempts[0] >= 3:
            messagebox.showerror("Locked", "Too many attempts!")
            root.destroy()

def toggle_show():
    if pass_entry.cget('show') == '*':
        pass_entry.config(show='')
        show_btn.config(text='Hide')

```



```

else:
    pass_entry.config(show='')
    show_btn.config(text='Show')

attempts = [0]
root = tk.Tk()
root.title("Login Form"); root.geometry("380x280")
root.config(bg="#1a237e"); root.resizable(False, False)

card = tk.Frame(root, bg="white", bd=0)
card.place(x=30, y=30, width=320, height=210)

tk.Label(card, text="USER LOGIN", font=("Arial",13,"bold"),
         bg="white", fg="#1a237e").place(x=100, y=15)
tk.Label(card, text="Username:", bg="white").place(x=20, y=65)
user_entry = tk.Entry(card, width=22, font=("Arial",10))
user_entry.place(x=110, y=63)

tk.Label(card, text="Password:", bg="white").place(x=20, y=105)
pass_entry = tk.Entry(card, width=17, show="*", font=("Arial",10))
pass_entry.place(x=110, y=103)
show_btn = tk.Button(card, text="Show", command=toggle_show, width=4, font=("Arial",8))
show_btn.place(x=265, y=103)

tk.Button(card, text="LOGIN", command=login, bg="#1a237e", fg="white",
         font=("Arial",11,"bold"), width=16).place(x=80, y=155)
root.bind('<Return>', lambda e: login())
root.mainloop()

```

Output: Dark blue window with white card. Username/password fields. Show/Hide toggle. Enter key also submits. 3 attempt lockout.

Q21. Draw shapes on Canvas (rectangle, line, oval).

```

import tkinter as tk
import math

root = tk.Tk()
root.title("Canvas Shapes Demo")
root.geometry("620x480")

canvas = tk.Canvas(root, width=600, height=420, bg="white",
                   relief="sunken", bd=2)
canvas.pack(padx=10, pady=10)

# 1. Rectangle
canvas.create_rectangle(20, 20, 160, 100,
                       fill="#bbdefb", outline="#1565c0", width=2)
canvas.create_text(90, 115, text="Rectangle", font=("Arial",9), fill="#1565c0")

# 2. Oval / Ellipse
canvas.create_oval(190, 20, 370, 110,
                  fill="#c8e6c9", outline="#2e7d32", width=2)
canvas.create_text(280, 120, text="Oval", font=("Arial",9), fill="#2e7d32")

# 3. Circle
canvas.create_oval(390, 20, 500, 130,
                  fill="#ffe0b2", outline="#e65100", width=2)
canvas.create_text(445, 140, text="Circle", font=("Arial",9), fill="#e65100")

# 4. Line (simple)
canvas.create_line(20, 175, 180, 175, fill="#c62828", width=3)
canvas.create_text(100, 190, text="Line", font=("Arial",9), fill="#c62828")

# 5. Diagonal line

```

```

canvas.create_line(20, 215, 180, 280, fill="#6a1b9a", width=2, dash=(5,3))
canvas.create_text(100, 290, text="Dashed Line", font=("Arial",9), fill="#6a1b9a")

# 6. Arc
canvas.create_arc(210, 160, 380, 290,
    start=0, extent=270, fill="#b3e5fc",
    outline="#0277bd", width=2)
canvas.create_text(295, 300, text="Arc", font=("Arial",9), fill="#0277bd")

# 7. Triangle (polygon)
canvas.create_polygon(430, 160, 380, 280, 480, 280,
    fill="#f8bbd0", outline="#880e4f", width=2)
canvas.create_text(430, 295, text="Triangle", font=("Arial",9), fill="#880e4f")

# 8. Star
def star(cx, cy, r_out, r_in, n=5):
    pts = []
    for i in range(2*n):
        r = r_out if i%2==0 else r_in
        a = math.radians(90 - i*180/n)
        pts.extend([cx + r*math.cos(a), cy - r*math.sin(a)])
    return pts

canvas.create_polygon(star(100, 355, 55, 22),
    fill="#fff9c4", outline="#f57f17", width=2)
canvas.create_text(100, 415, text="Star", font=("Arial",9), fill="#f57f17")

# 9. Nested rectangles
for i, col in enumerate(["#e3f2fd", "#bbdefb", "#90caf9"]):
    canvas.create_rectangle(310-i*12, 320-i*12,
        470+i*12, 400+i*12,
        fill=col, outline="#1565c0", width=1)
canvas.create_text(390, 415, text="Nested Rect", font=("Arial",9), fill="#1565c0")

root.mainloop()

```

Output: Canvas displays: Rectangle, Oval, Circle, Line, Dashed Line, Arc, Triangle, Star, Nested Rectangles – all colored and labeled.

Q22. Use Listbox and Radiobutton in a form.

```

import tkinter as tk
from tkinter import messagebox

root = tk.Tk()
root.title("Listbox & Radiobutton Demo")
root.geometry("460px400".replace("px", "x"))
root.config(bg="#e8eaf6")

tk.Label(root, text="Student Course Selection Form",
    font=("Arial",13,"bold"), bg="#e8eaf6", fg="#1a237e").pack(pady=10)

# ■■ LISTBOX section ■■
lf = tk.LabelFrame(root, text="Select Subjects", bg="#e8eaf6", font=("Arial",10))
lf.pack(padx=20, pady=5, fill="x")

subjects = ["Mathematics", "Physics", "Chemistry", "Computer Science",
    "English", "Biology", "Economics", "History"]
scrollbar = tk.Scrollbar(lf)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
listbox = tk.Listbox(lf, selectmode=tk.MULTIPLE, height=5,
    yscrollcommand=scrollbar.set, font=("Arial",10))
for subj in subjects:
    listbox.insert(tk.END, subj)

```

```

listbox.pack(side=tk.LEFT, fill="x", expand=True, padx=5, pady=5)
scrollbar.config(command=listbox.yview)

# ■■ RADIOBUTTON section ■■
rf = tk.LabelFrame(root, text="Select Stream", bg="#e8eaf6", font=("Arial",10))
rf.pack(padx=20, pady=5, fill="x")

stream_var = tk.StringVar(value="Science")
for stream in ["Science", "Commerce", "Arts", "Engineering"]:
    tk.Radiobutton(rf, text=stream, variable=stream_var, value=stream,
                  bg="#e8eaf6", font=("Arial",10)).pack(side=tk.LEFT, padx=12)

def submit():
    selected_idx = listbox.curselection()
    selected_subj = [listbox.get(i) for i in selected_idx]
    stream = stream_var.get()
    if not selected_subj:
        messagebox.showwarning("Select", "Please select at least one subject!")
        return
    result = f"Stream: {stream}\nSubjects: {' '.join(selected_subj)}"
    messagebox.showinfo("Your Selection", result)

tk.Button(root, text="Submit Selection", command=submit,
          bg="#1a237e", fg="white", font=("Arial",11)).pack(pady=12)

root.mainloop()

```

Output: Form with multi-select Listbox (8 subjects) and Radiobuttons (4 streams). Submit shows chosen stream and subjects in messagebox.

Q23. Connect to MySQL database and create a table.

```

import mysql.connector
from mysql.connector import Error

try:
    # Step 1: Connect to MySQL server
    conn = mysql.connector.connect(
        host="localhost", user="root", password="yourpassword"
    )
    print(f"Connected to MySQL v{conn.get_server_info()}")
    cursor = conn.cursor()

    # Step 2: Create database
    cursor.execute("CREATE DATABASE IF NOT EXISTS college_db")
    cursor.execute("USE college_db")
    print("Database 'college_db' selected.")

    # Step 3: Create student table
    cursor.execute(
        "CREATE TABLE IF NOT EXISTS student ("
        " id          INT PRIMARY KEY AUTO_INCREMENT, "
        " roll_no     VARCHAR(20) UNIQUE NOT NULL, "
        " name        VARCHAR(100) NOT NULL, "
        " branch      VARCHAR(50), "
        " semester    INT, "
        " percentage  FLOAT, "
        " email        VARCHAR(100) UNIQUE, "
        " created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP"
        ") "
    )
    conn.commit()
    print("Table 'student' created successfully!")

    # Step 4: Verify table structure

```

```

cursor.execute("DESCRIBE student")
print("\nTable Structure:")
print(f"{'Field':<15} {'Type':<20} {'Null':<6} {'Key':<6} {'Default'}")
print("-" * 60)
for row in cursor.fetchall():
    print(f"{row[0]:<15} {row[1]:<20} {row[2]:<6} {row[3]:<6} {str(row[4])}")

except Error as e:
    print(f"Error: {e}")
finally:
    if conn.is_connected():
        cursor.close(); conn.close()
        print("\nConnection closed.")

```

Output: Connected to MySQL v8.0.32 Database 'college_db' selected. Table 'student' created successfully! Table Structure: Field Type Null Key Default

```

----- id int NO PRI None roll_no varchar(20)
NO UNI None name varchar(100) NO None branch varchar(50) YES None semester int YES None percentage
float YES None email varchar(100) YES UNI None created_at timestamp YES CURRENT_TIMESTAMP Connection
closed.

```

Q24. Insert and retrieve records.

```

import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root",
    password="yourpassword", database="college_db"
)
cursor = conn.cursor()

# ■■■ INSERT ■■■
students = [
    ('R001', 'Rahul Sharma', 'CS', 5, 87.5, 'rahul@college.edu'),
    ('R002', 'Priya Patel', 'Electronics', 3, 91.2, 'priya@college.edu'),
    ('R003', 'Amit Desai', 'Mechanical', 7, 78.9, 'amit@college.edu'),
    ('R004', 'Sneha Kulkarni', 'CS', 5, 95.0, 'sneha@college.edu'),
]

sql_insert = (
    "INSERT INTO student (roll_no,name,branch,semester,percentage,email)"
    " VALUES (%s,%s,%s,%s,%s,%s)"
)

cursor.executemany(sql_insert, students)
conn.commit()
print(f"{cursor.rowcount} records inserted.")

# ■■■ RETRIEVE ALL ■■■
print("\n=== All Students ===")
cursor.execute("SELECT roll_no, name, branch, percentage FROM student")
rows = cursor.fetchall()
print(f"{'Roll':<8} {'Name':<18} {'Branch':<14} {'%'}")
print("-" * 48)
for r in rows:
    print(f"{r[0]:<8} {r[1]:<18} {r[2]:<14} {r[3]}")

# ■■■ RETRIEVE with WHERE ■■■
print("\n=== CS Students ===")
cursor.execute("SELECT name, percentage FROM student WHERE branch=%s", ("CS",))
for r in cursor.fetchall():
    print(f" {r[0]}: {r[1]}%")

# ■■■ RETRIEVE with ORDER BY ■■■
print("\n=== Top 2 by Marks ===")
cursor.execute("SELECT name, percentage FROM student ORDER BY percentage DESC LIMIT 2")

```

```

for i, r in enumerate(cursor.fetchall(), 1):
    print(f" {i}. {r[0]} - {r[1]}%")

# ■■ fetchone() demo ■■
cursor.execute("SELECT * FROM student WHERE roll_no=%s", ("R002",))
one = cursor.fetchone()
print(f"\nfetchone() -> R002: {one[2]} | {one[3]} | {one[5]}%")

cursor.close(); conn.close()

```

```

Output: 4 records inserted. === All Students === Roll Name Branch %
----- R001 Rahul Sharma CS 87.5 R002 Priya Patel
Electronics 91.2 R003 Amit Desai Mechanical 78.9 R004 Sneha Kulkarni CS 95.0 === CS Students === Rahul
Sharma: 87.5% Sneha Kulkarni: 95.0% === Top 2 by Marks === 1. Sneha Kulkarni - 95.0% 2. Priya Patel -
91.2% fetchone() -> R002: Priya Patel | Electronics | 91.2%

```

Q25. Update and delete records in student table.

```

import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root",
    password="yourpassword", database="college_db"
)
cursor = conn.cursor()

# ■■ UPDATE single record ■■
cursor.execute(
    "UPDATE student SET percentage=%s WHERE roll_no=%s",
    (93.0, "R001")
)
conn.commit()
print(f"UPDATE: {cursor.rowcount} row(s) affected (R001 -> 93.0%)")

# ■■ UPDATE multiple fields ■■
cursor.execute(
    "UPDATE student SET semester=%s, percentage=%s WHERE roll_no=%s",
    (6, 96.0, "R004")
)
conn.commit()
print(f"UPDATE: R004 -> semester=6, percentage=96.0%")

# ■■ UPDATE with condition ■■
cursor.execute(
    "UPDATE student SET percentage = percentage + 2 WHERE branch=%s",
    ("CS",)
)
conn.commit()
print(f"BULK UPDATE: {cursor.rowcount} CS student(s) got +2% bonus")

# ■■ Verify updates ■■
cursor.execute("SELECT roll_no, name, percentage FROM student ORDER BY percentage DESC")
print("\nAfter Updates:")
for r in cursor.fetchall():
    print(f" {r[0]}: {r[1]} - {r[2]}%")

# ■■ DELETE specific record ■■
cursor.execute("DELETE FROM student WHERE roll_no=%s", ("R003",))
conn.commit()
print(f"\nDELETE: R003 removed ({cursor.rowcount} row affected)")

# ■■ Safe delete (check first) ■■
roll = "R999"
cursor.execute("SELECT * FROM student WHERE roll_no=%s", (roll,))

```

```

if cursor.fetchone():
    cursor.execute("DELETE FROM student WHERE roll_no=%s", (roll,))
    conn.commit()
    print(f"Deleted {roll}")
else:
    print(f"Safe delete: {roll} not found, nothing deleted.")

```

```

# ■■ Final count ■■
cursor.execute("SELECT COUNT(*) FROM student")
print(f"\nRemaining records: {cursor.fetchone()[0]}")

```

```

cursor.close(); conn.close()

```

```

Output: UPDATE: 1 row(s) affected (R001 -> 93.0%) UPDATE: R004 -> semester=6, percentage=96.0% BULK
UPDATE: 2 CS student(s) got +2% bonus After Updates: R004: Sneha Kulkarni - 98.0% R002: Priya Patel -
91.2% R001: Rahul Sharma - 95.0% R003: Amit Desai - 78.9% DELETE: R003 removed (1 row affected) Safe
delete: R999 not found, nothing deleted. Remaining records: 3

```

Section C: Django Student Management (Q26–Q27)

Q26. Create a Django project and app for student management.

File: Setup Commands

```

django-admin startproject StudentMgmt
cd StudentMgmt
python manage.py startapp students
# Add 'students' to INSTALLED_APPS in settings.py

```

File: students/models.py

```

from django.db import models

class Student(models.Model):
    BRANCH_CHOICES = [
        ('CS', 'Computer Science'),
        ('ECE', 'Electronics'),
        ('ME', 'Mechanical'),
        ('CE', 'Civil'),
    ]
    roll_no = models.CharField(max_length=20, unique=True)
    name = models.CharField(max_length=100)
    branch = models.CharField(max_length=5, choices=BRANCH_CHOICES)
    semester = models.IntegerField(choices=[(i, f"Sem {i}") for i in range(1,9)])
    percentage = models.FloatField()
    email = models.EmailField(unique=True)
    phone = models.CharField(max_length=10, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.roll_no} - {self.name}"

    def get_grade(self):
        if self.percentage >= 90: return 'A+'
        elif self.percentage >= 75: return 'A'
        elif self.percentage >= 60: return 'B'
        return 'C'

    class Meta:
        ordering = ['-percentage']

```

File: students/views.py

```

from django.shortcuts import render, redirect, get_object_or_404
from .models import Student

def index(request):
    total = Student.objects.count()

```

```

toppers = Student.objects.filter(percentage__gte=75).count()
avg      = sum(s.percentage for s in Student.objects.all()) / total if total else 0
return render(request, 'students/index.html',
               {'total': total, 'toppers': toppers, 'avg': round(avg, 2)})

def list_students(request):
    branch = request.GET.get('branch', '')
    students = Student.objects.filter(branch=branch) if branch else Student.objects.all()
    return render(request, 'students/list.html',
                  {'students': students, 'selected': branch})

def add_student(request):
    if request.method == 'POST':
        Student.objects.create(
            roll_no = request.POST['roll_no'],
            name     = request.POST['name'],
            branch  = request.POST['branch'],
            semester = request.POST['semester'],
            percentage = request.POST['percentage'],
            email    = request.POST['email'],
            phone    = request.POST.get('phone', ''),
        )
        return redirect('list_students')
    return render(request, 'students/add.html')

def delete_student(request, pk):
    stu = get_object_or_404(Student, pk=pk)
    stu.delete()
    return redirect('list_students')

```

File: students/urls.py

```

from django.urls import path
from . import views

app_name = 'students'
urlpatterns = [
    path('', views.index, name='index'),
    path('list/', views.list_students, name='list_students'),
    path('add/', views.add_student, name='add_student'),
    path('delete/<int:pk>/', views.delete_student, name='delete_student'),
]

```

File: students/admin.py

```

from django.contrib import admin
from models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ('roll_no', 'name', 'branch', 'semester', 'percentage', 'get_grade')
    list_filter = ('branch', 'semester')
    search_fields = ('name', 'roll_no', 'email')

```

File: Setup DB

```

python manage.py makemigrations students
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver

```

Output: / -> Dashboard with stats /list/ -> All students with branch filter /add/ -> Add form /admin/ -> Full management

Q27. Display a webpage using templates (use 2 templates — base + child).

File: templates/base.html (Template 1: Parent/Base)

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}Student Management{% endblock %}</title>
  <style>
    * { box-sizing:border-box; margin:0; padding:0; font-family:Arial,sans-serif; }
    body { background:#e8eaf6; }
    header {
      background:#1a237e; color:white; padding:16px 30px;
      display:flex; justify-content:space-between; align-items:center;
    }
    header h1 { font-size:22px; }
    nav a { color:#aab6fb; text-decoration:none; margin-left:18px; font-size:14px; }
    nav a:hover { color:white; }
    .container { max-width:1000px; margin:30px auto; padding:0 20px; }
    footer {
      text-align:center; padding:16px; background:#1a237e;
      color:#aab6fb; margin-top:40px; font-size:13px;
    }
  </style>
  {% block extra_css %}{% endblock %}
</head>
<body>
  <header>
    <h1>Student Management System</h1>
    <nav>
      <a href="{% url 'students:index' %}">Home</a>
      <a href="{% url 'students:list_students' %}">Students</a>
      <a href="{% url 'students:add_student' %}">Add Student</a>
      <a href="/admin/">Admin</a>
    </nav>
  </header>

  <div class="container">
    {% block content %}
    <!-- Child templates fill this block -->
    {% endblock %}
  </div>

  <footer>
    &copy; 2026 Student Management System | Built with Django
  </footer>
  {% block extra_js %}{% endblock %}
</body>
</html>

```

File: templates/students/list.html (Template 2: Child inheriting base)

```

{% extends 'base.html' %}

{% block title %}Student List{% endblock %}

{% block extra_css %}
<style>
  .filter-bar { background:white; padding:15px; border-radius:8px; margin-bottom:20px; }
  table { width:100%; border-collapse:collapse; background:white; border-radius:8px;
    overflow:hidden; box-shadow:0 2px 8px rgba(0,0,0,0.1); }
  th { background:#1a237e; color:white; padding:12px; text-align:left; }
  td { padding:11px 12px; border-bottom:1px solid #e0e0e0; }
  tr:hover { background:#f5f5f5; }
  .btn { padding:5px 12px; border-radius:4px; text-decoration:none; font-size:13px; }
  .btn-del { background:#c62828; color:white; }
  .grade-A { color:#2e7d32; font-weight:bold; }
  .grade-B { color:#1565c0; font-weight:bold; }

```



```

        .grade-C { color:#e65100; font-weight:bold; }
</style>
{% endblock %}

{% block content %}
<h2 style="margin-bottom:16px; color:#1a237e">All Students</h2>

<div class="filter-bar">
    <form method="get" style="display:flex; gap:10px; align-items:center">
        <label>Filter by Branch:</label>
        <select name="branch" onchange="this.form.submit()">
            <option value="">All Branches</option>
            {% for code, label in branch_choices %}
                <option value="{{ code }}" {% if selected == code %}selected{% endif %}>
                    {{ label }}
                </option>
            {% endfor %}
        </select>
        <a href="{% url 'students:add_student' %}"
            style="background:#2e7d32;color:white;padding:7px 16px;
                border-radius:5px;text-decoration:none;margin-left:auto">
            + Add Student
        </a>
    </form>
</div>

{% if students %}
<table>
    <tr><th>#</th><th>Roll No</th><th>Name</th><th>Branch</th>
        <th>Sem</th><th>%</th><th>Grade</th><th>Action</th></tr>
    {% for s in students %}
    <tr>
        <td>{{ forloop.counter }}</td>
        <td>{{ s.roll_no }}</td>
        <td>{{ s.name }}</td>
        <td>{{ s.get_branch_display }}</td>
        <td>{{ s.semester }}</td>
        <td>{{ s.percentage }}</td>
        <td class="grade-{{ s.get_grade|slice:':1' }}">{{ s.get_grade }}</td>
        <td>
            <a href="{% url 'students:delete_student' s.pk %}"
                class="btn btn-del"
                onclick="return confirm('Delete {{ s.name }}?')">Delete</a>
        </td>
    </tr>
    {% endfor %}
</table>
<p style="margin-top:10px;color:#555">Total: {{ students|length }} student(s)</p>
{% else %}
    <div style="text-align:center;padding:40px;background:white;border-radius:8px">
        <h3 style="color:#888">No students found.</h3>
        <a href="{% url 'students:add_student' %}" style="color:#1a237e">Add first student</a>
    </div>
{% endif %}
{% endblock %}

```

Note: Template 1 (base.html) defines the layout with header, nav, footer and content block. Template 2 (list.html) uses extends and block tags to inherit and override only the content section — avoiding code duplication.

Output: Browser shows professional page with shared header/footer from base.html and student table from list.html.